EVO AURA

41 Python Files and Application Flow

New file list, responsibilities, navigation map, and full program flow for Codex.

Document Type	PDF 2 of 2 - New file list and flow
Target OS	Windows 10 and later only. Do not add Windows 7/8 compatibility changes.
Purpose	Codex-ready instructions for refactoring, packaging, and smooth EXE deployment.
DB Rule	Each computer creates and uses its own local SQLite DB on first run.

Important instruction for Codex: keep the software working while refactoring. Use compatibility imports where needed, test after every file move, and never hardcode user data inside Program Files or the installation folder.

1. Final 41 Python file list

This is the clean target structure. The goal is not to create empty files only; each file must own one clear responsibility so Evo Aura is easier to edit, package, and maintain.

No.	File	Description / responsibility
1	main.py	Application entry point. Creates QApplication, applies theme/icon/input guard, starts boot flow.
2	core/app_shell.py	MainWindow shell after login. Hosts sidebar, QStackedWidget, route loading, logout.
3	core/sidebar.py	Collapsible sidebar and NAV_ITEMS. Navigation keys must match core/routes.py.
4	core/routes.py	Single route registry. Maps nav keys to page module, class name, and lazy loader.
5	core/theme.py	Single color, font, spacing, and stylesheet source for the whole app.
6	core/ui_helpers.py	Reusable labels, buttons, cards, inputs, toasts, table helpers, empty states.
7	core/db.py	DB connection manager. Stores DB path in LocalAppData, provides safe connection helpers.
8	core/migrations.py	Creates/updates all tables and indexes. Idempotent migrations only.
9	core/auth_service.py	Password hashing, TOTP verification, recovery codes, user role checks.
10	core/input_behavior.py	Global input behavior and keyboard guard for PyQt6.
11	core/app_branding.py	App icon, window icon, resource path helpers for normal run and PyInstaller.
12	pages/auth/ask_db_page.py	First-run company/workspace DB name screen.
13	pages/auth/master_auth_page.py	Master TOTP verification before new DB creation.
14	pages/auth/company_setup_page.py	Company profile setup: name, phone, address, GST, logo, invoice footer.
15	pages/auth/signup_page.py	Create ADMIN first, then optional regular users.
16	pages/auth/qr_setup_page.py	Show authenticator QR and recovery codes.
17	pages/auth/login_page.py	Username/password login screen.
18	pages/auth/otp_page.py	TOTP and recovery code verification screen.
19	pages/dashboard_page.py	Main dashboard/KPI page after login.
20	pages/sales/new_sale_page.py	POS billing page: barcode/search, cart, discount, payment, print.
21	pages/sales/returns_page.py	Sales return workflow, refund/credit, stock restore.
22	pages/sales/bill_view_page.py	Find, view, reprint, export, and cancel bills.
23	pages/inventory/product_page.py	Product master, variants, barcode, pricing, stock summary.
24	pages/inventory/purchase_invoices_page.py	Purchase invoice entry, supplier invoice, stock increase.
25	pages/inventory/supplier_page.py	Supplier master, purchase history, balances, contact details.
26	pages/inventory/reorder_control_page.py	Low stock/reorder alerts and reorder planning.
27	pages/customers/customer_center_page.py	Customer master, history, contact, ledger view.
28	pages/customers/due_collection_page.py	Credit/due collection payments and pending balances.
29	pages/customers/loyalty_page.py	Loyalty points, rewards, membership tiers.
30	pages/finance/pl_summary_page.py	Profit and loss summary using sales, purchases, expense data.
31	pages/finance/expense_tracking_page.py	Expense entry, categories, recurring expenses, attachments.
32	pages/finance/gst_breakdown_page.py	GST output/input breakdown and tax summary.
33	pages/finance/cashflow_page.py	Cash in/out flow from sales, purchase payments, expenses, dues.
34	pages/reports/daybook_page.py	Daily transaction book for sales, purchases, receipts, expenses.

No.	File	Description / responsibility
35	pages/reports/stock_report_page.py	Stock valuation, movement, ageing, item-wise stock reports.
36	pages/reports/trial_balance_page.py	Accounting-style trial balance summary.
37	pages/reports/export_page.py	Export reports to PDF/Excel/CSV; backup and print actions.
38	pages/settings/company_settings_page.py	Edit company profile, GST, logo, invoice footer.
39	pages/settings/users_roles_page.py	Manage users, roles, permissions, reset recovery codes.
40	pages/settings/security_page.py	2FA settings, master key policy, backup/restore security controls.
41	pages/settings/audit_log_page.py	Audit trail for login, billing, edits, deletes, exports.

2. Required package tree

Create this exact tree. Non-Python assets are included for context but not counted in the 41 Python files.

```
evo_aura/
 main.py
 core/
 __init__.py
 app_shell.py
 sidebar.py
 routes.py
 theme.py
 ui_helpers.py
 db.py
 migrations.py
 auth_service.py
 input_behavior.py
 app_branding.py
 pages/
 __init__.py
 auth/
 __init__.py
 ask_db_page.py
 master_auth_page.py
 company_setup_page.py
 signup_page.py
 qr_setup_page.py
 login_page.py
 otp_page.py
 sales/
 __init__.py
 new_sale_page.py
 returns_page.py
 bill_view_page.py
 inventory/
 __init__.py
 product_page.py
 purchase_invoices_page.py
 supplier_page.py
 reorder_control_page.py
 customers/
 __init__.py
 customer_center_page.py
 due_collection_page.py
 loyalty_page.py
 finance/
 __init__.py
 pl_summary_page.py
 expense_tracking_page.py
 gst_breakdown_page.py
 cashflow_page.py
 reports/
 __init__.py
 daybook_page.py
 stock_report_page.py
 trial_balance_page.py
```

```
 export_page.py
 settings/
 __init__.py
 company_settings_page.py
 users_roles_page.py
 security_page.py
 audit_log_page.py
 assets/
 icons/
 templates/
 installer/
 tests/
```

3. Application boot flow

Codex must keep this flow. This is important because after installation as EXE, every computer should create and use its own local DB.

```
main.py
 - create QApplication
 - apply theme and app icon
 - call ensure_global_input_guard once
 - create BootController or MainController
 - find config at %LOCALAPPDATA%\EvoAura\config.json
 - if DB missing: AskDB -> MasterAuth -> create DB -> migrations -> CompanySetup -> Signup -> QRSetup
-> Login -> OTP -> AppShell
 - if DB exists and users missing: Signup -> QRSetup -> Login -> OTP -> AppShell
 - if DB exists and users exist: Login -> OTP -> AppShell

AppShell
 - load core/sidebar.py
 - host QStackedWidget
 - lazy-load pages using core/routes.py
 - pass app context to every page: db path, user, company, permissions, services
 - handle logout, company settings refresh, and safe close
```

4. Sidebar navigation map

The sidebar key, route key, class name, and target file must match. If existing sidebar uses old keys like pl_summarys or securitys, Codex must update sidebar and routes together or keep aliases.

Route key	Class name	File
home	DashboardPage	pages/dashboard_page.py
new_sale	NewSalePage	pages/sales/new_sale_page.py
returns	ReturnsPage	pages/sales/returns_page.py
bill_view	BillViewPage	pages/sales/bill_view_page.py
product	ProductPage	pages/inventory/product_page.py
purchase_orders	PurchaseInvoicesPage	pages/inventory/purchase_invoices_page.py
suppliers	SupplierPage	pages/inventory/supplier_page.py
low_stock	ReorderControlPage	pages/inventory/reorder_control_page.py
customer	CustomerCenterPage	pages/customers/customer_center_page.py
credit	DueCollectionPage	pages/customers/due_collection_page.py
loyalty	LoyaltyPage	pages/customers/loyalty_page.py
pl_summary	PLSummaryPage	pages/finance/pl_summary_page.py
expense	ExpenseTrackingPage	pages/finance/expense_tracking_page.py
gst	GSTBreakdownPage	pages/finance/gst_breakdown_page.py
cashflow	CashflowPage	pages/finance/cashflow_page.py

Route key	Class name	File
daybook	DaybookPage	pages/reports/daybook_page.py
stock_reports	StockReportPage	pages/reports/stock_report_page.py
trial_balance	TrialBalancePage	pages/reports/trial_balance_page.py
export	ExportPage	pages/reports/export_page.py
company	CompanySettingsPage	pages/settings/company_settings_page.py
users_roles	UsersRolesPage	pages/settings/users_roles_page.py
security	SecurityPage	pages/settings/security_page.py
audit	AuditLogPage	pages/settings/audit_log_page.py

5. Page flow by business section

Section	Flow
Dashboard	After login, show KPIs: today sales, monthly sales, stock value, low stock count, credit due, GST summary. Data should refresh on demand and not recalculate heavy reports on every paint.
Sales - New Sale	Scan/search product -> select variant -> add to cart -> edit quantity/price/discount -> choose payment/cash/credit -> save sale transaction -> update stock -> generate invoice -> print or export.
Sales - Returns	Search invoice -> choose return items -> calculate refund/credit -> restore stock if item is saleable -> save return -> audit log.
Sales - Bill View	Search by invoice/date/customer -> open bill -> reprint/export -> show payment and return status.
Inventory - Product	Create product -> variants for size/color/fabric -> barcode/SKU -> prices -> reorder level -> stock summary. Use variant-level stock for textile colors and sizes.
Inventory - Purchase Invoices	Select supplier -> add purchased products/variants -> enter cost, GST, discount -> save invoice -> increase stock -> update supplier balance.
Inventory - Supplier	Supplier master -> contact details -> ledger -> purchase history -> payment status.
Inventory - Reorder Control	Show low stock and fast moving items -> create purchase suggestion -> optionally send/export reorder list.
Customers - Center	Add/search customer -> sales history -> ledger -> due balance -> loyalty status.
Customers - Due Collection	View pending credit -> collect full/partial payment -> update ledger -> print receipt.
Customers - Loyalty	Configure points -> award on sale -> redeem -> show customer loyalty history.
Finance	P&L, expense, GST, and cashflow pages use saved transactions. Heavy calculations should run in worker threads or cached queries.
Reports	Day book, stock report, trial balance, and exports should use filters, pagination, and background export jobs.
Settings	Company settings, users/roles, security, and audit log. Security changes must be logged.

6. Core module flow

Core file	How it works in the full flow
core/theme.py	Defines C dict/colors, fonts, common sizes, global stylesheet, and accent color. All pages import from here, not hardcoded colors.
core/ui_helpers.py	Exports common widgets such as GradientButton, StyledInput, Toast, KPI cards, section headers, empty state, and table actions.
core/db.py	Provides get_app_data_dir(), get_db_path(), connect(), transaction(), and current DB context. Does not create UI.
core/migrations.py	Runs CREATE TABLE IF NOT EXISTS, ALTER TABLE checks, and indexes. Called after DB selection/creation before any page loads.

Core file	How it works in the full flow
core/auth_service.py	Contains hash_password(), verify_password(), generate_totp_secret(), verify_totp(), generate_recovery_codes(), consume_recovery_code().
core/routes.py	Loads pages by key only when needed. Prevents startup lag.
core/app_shell.py	Main logged-in window. Owns sidebar and stacked pages. No auth page code here.
core/sidebar.py	Only navigation UI. Emits route keys. Does not import heavy page modules.
core/app_branding.py	Handles icons/assets both from source and PyInstaller using resource_path().
core/input_behavior.py	Applies global input behavior once. No page-specific logic here.

7. Existing files to new file compatibility mapping

Keep compatibility wrappers until all imports are updated. This prevents sudden ImportError while Codex moves files.

Existing name	Recommended compatibility handling
evoaura_app.py	Keep as: from main import main; if __name__ == "__main__": main(). Remove only after shortcuts and build scripts use main.py.
sidebar.py	After moving to core/sidebar.py, either update all imports or leave sidebar.py wrapper: from core.sidebar import Sidebar, NAV_ITEMS.
dashboard.py	Leave wrapper from pages.dashboard_page import DashboardPage if older code imports dashboard.
product_page.py	Leave wrapper from pages.inventory.product_page import ProductPage until imports are updated.
supplier_page.py	Leave wrapper from pages.inventory.supplier_page import SupplierPage.
purchase_orders_page.py	Leave wrapper from pages.inventory.purchase_invoices_page import PurchaseInvoicesPage. Add old class alias if needed.
low_stock_page.py	Leave wrapper from pages.inventory.reorder_control_page import ReorderControlPage.
customer_page.py	Leave wrapper from pages.customers.customer_center_page import CustomerCenterPage.
credit_management_page.py	Leave wrapper from pages.customers.due_collection_page import DueCollectionPage.
loyalty_page.py	Leave wrapper from pages.customers.loyalty_page import LoyaltyPage.

8. EXE flow after refactor

The EXE should behave exactly like source run, except files are loaded from bundled assets and data is written to LocalAppData.

```
Installed EXE starts
 - App reads bundled assets using resource_path()
 - App reads/writes data using LocalAppData, not install folder
 - If no DB config: show first-run DB setup flow
 - If DB config exists: validate DB path and run migrations
 - Login and OTP
 - AppShell and lazy page routes

Installer includes
 - EvoAura.exe
 - assets/icons
 - assets/templates if required
 - no __pycache__, no .git, no sample DB as production DB
 - optional sample DB only in tests/demo folder, not active runtime
```

9. Build and installer commands

Use one-folder first for testing. It is smoother for PyQt apps because DLLs and plugins remain visible and startup is often faster than one-file extraction.

```
# 1. Clean venv on Windows 10/11
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install PyQt6 pyotp "qrcode[pil]" Pillow pyinstaller

# 2. Source test
python main.py

# 3. PyInstaller one-folder test
pyinstaller --noconfirm --clean --windowed --name EvoAura ^
 --icon assets\icons\app.ico ^
 --add-data "assets\icons;assets\icons" ^
 --add-data "assets\templates;assets\templates" ^
 --hidden-import PyQt6.QtSvg ^
 main.py

# 4. Run test
.dist\EvoAura\EvoAura.exe

# 5. Create installer after dist test works
# Use Inno Setup: include dist\EvoAura\* and create shortcuts.
```

10. Performance definition of done

- Cold start to login should be fast because only auth modules load before login.
- Opening dashboard should not instantiate all pages.
- Search boxes should debounce DB queries and not query on every key press immediately.
- Product and bill tables should use pagination or model/view rendering for large records.
- Exports/backups/report generation should not freeze the UI.
- Icons and logos must load in source mode and EXE mode.
- Fresh install on Windows 10/11 must create DB through StartAuth/AskDB flow without admin permissions.

11. Codex prompt for this file-flow PDF

```
Create/refactor my Evo Aura PyQt6 billing software into exactly the 41 Python files listed in this
PDF. Implement the boot flow, auth flow, sidebar route flow, and first-run per-computer DB creation
exactly as described. Keep Windows 10 and later as the only target. Do not add Windows 7/8
compatibility changes. Keep existing file wrappers temporarily so old imports do not break. Use
LocalAppData for DB/log/export/backups, resource_path for bundled assets, lazy page loading for
performance, and PyInstaller plus Inno Setup for the final installer.
```